```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, GRU, Conv1D,
Flatten, Concatenate, Reshape, Dropout
from tensorflow.keras.optimizers import Adam

# Step 2: Load and Preprocess Data
df = pd.read_csv('Historical_Banking_Data.csv', low_memory=False)
df = df.sample(frac=0.3, random_state=42)  # use 30% of data for
faster training
df.drop(['ID', 'Customer_ID', 'Name', 'SSN', 'Type_of_Loan',
'Payment_Behaviour', 'Credit_History_Age'], axis=1, inplace=True)
df = df.dropna()

cat_cols = ['Month', 'Occupation', 'Credit_Mix',
'Payment_of_Min_Amount', 'Credit_Score']
label_encoders = {col: LabelEncoder() for col in cat_cols}
for col in cat_cols:
    df[col] = label_encoders[col].fit_transform(df[col])

df = df[~df['Age'].astype(str).str.contains('-|_', regex=True)]
df = df[df['Age'].astype(str).str.isnumeric()]
df['Age'] = df['Age'].astype(int)

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = pd.to_numeric(df[col], errors='coerce')
df = df.dropna()

lln_features = ['Month', 'Occupation', 'Credit_Mix',
'Payment_of_Min_Amount', 'Age', 'Interest_Rate',
'Delay_from_due_date']
cnn_features = ['Annual_Income', 'Monthly_Inhand_Salary',
'Outstanding_Debt', 'Credit_Utilization_Ratio',
                'Total_EMI_per_month', 'Amount_invested_monthly',
'Monthly_Balance']

X_lln = df[lln_features]
X_cnn = df[cnn_features]
y = df['Credit_Score']

scaler_lln = StandardScaler()
scaler_cnn = StandardScaler()

X_lln_scaled = scaler_lln.fit_transform(X_lln)
X_cnn_scaled = scaler_cnn.fit_transform(X_cnn)
```

```
X_lln_train, X_lln_test, X_cnn_train, X_cnn_test, y_train, y_test =
train_test_split(
    X_lln_scaled, X_cnn_scaled, y, test_size=0.2, random_state=42)

cnn_numerical_data = df[cnn_features + ['Credit_Score']].copy()
cnn_numerical_data = pd.concat([

pd.DataFrame(scaler_cnn.transform(cnn_numerical_data[cnn_features]),
columns=cnn_features),
    cnn_numerical_data[['Credit_Score']].reset_index(drop=True)
], axis=1)
cnn_numerical_data.to_csv("cnn_numerical_tabular_dataset.csv",
index=False)

# Step 3: Honey Bee Optimizer Definition
def honey_bee_optimizer(obj_func, bounds, n_bees=5, n_elite=2,
n_iter=3, patch_size=0.1):
    dim = len(bounds)
    population = np.random.rand(n_bees, dim)
    for i in range(dim):
        min_b, max_b = bounds[i]
        population[:, i] = min_b + population[:, i] * (max_b - min_b)

    for _ in range(n_iter):
        fitness = np.array([obj_func(ind) for ind in population])
        sorted_idx = np.argsort(fitness)
        population = population[sorted_idx]

        for i in range(n_elite):
            for _ in range(int(n_bees/n_elite)-1):
                new_bee = population[i] + patch_size *
(np.random.rand(dim) - 0.5)
                for d in range(dim):
                    new_bee[d] = np.clip(new_bee[d], bounds[d][0],
bounds[d][1])
                population = np.vstack([population, new_bee])

        population = population[:n_bees]

    best_fitness = obj_func(population[0])
    return population[0], best_fitness

# Step 4: Train and Save Models for All Fusion Types (with
optimizations)
def build_and_train_model(params, fusion_type):
    gru_units = int(params[0])
    cnn_filters = int(params[1])
    cnn_kernel = int(params[2])
    dense_units = int(params[3])
    learning_rate = params[4]
```

```
    input_lln = Input(shape=(X_lln_train.shape[1],))
    x = Reshape((X_lln_train.shape[1], 1))(input_lln)
    x = GRU(gru_units, activation='tanh')(x)
    x = Dense(dense_units, activation='relu')(x)
    lln_output = Dense(16, activation='relu')(x)

    input_cnn = Input(shape=(X_cnn_train.shape[1],))
    y = Reshape((X_cnn_train.shape[1], 1))(input_cnn)
    y = Conv1D(cnn_filters, kernel_size=cnn_kernel, activation='relu')
(y)
    y = Flatten()(y)
    y = Dense(dense_units, activation='relu')(y)
    cnn_output = Dense(16, activation='relu')(y)

    if fusion_type == "early":
        combined_input = Concatenate()([input_lln, input_cnn])
        fusion = Dense(32, activation='relu')(combined_input)
        output = Dense(3, activation='softmax')(fusion)
        model = Model(inputs=[input_lln, input_cnn], outputs=output)
    elif fusion_type == "late":
        lln_pred = Dense(3, activation='softmax')(lln_output)
        cnn_pred = Dense(3, activation='softmax')(cnn_output)
        output = tf.keras.layers.Average()([lln_pred, cnn_pred])
        model = Model(inputs=[input_lln, input_cnn], outputs=output)
    else:  # intermediate
        fusion = Concatenate()([lln_output, cnn_output])
        fusion = Dense(32, activation='relu')(fusion)
        output = Dense(3, activation='softmax')(fusion)
        model = Model(inputs=[input_lln, input_cnn], outputs=output)

    model.compile(optimizer=Adam(learning_rate=learning_rate),
                  loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    model.fit([X_lln_train, X_cnn_train], y_train, epochs=3,
batch_size=128, validation_data=([X_lln_test, X_cnn_test], y_test))
    model.save(f"lln_cnn_{fusion_type}_fusion_model.h5")
    return model

# Step 5: Run HBO and Train Models for All Fusion Types

def objective_function(params, fusion_type):
    return build_and_train_model(params,
fusion_type).evaluate([X_lln_test, X_cnn_test], y_test, verbose=0)[1]
* -1

# Optimize once using intermediate fusion as baseline
best_params, best_score = honey_bee_optimizer(lambda p:
objective_function(p, "intermediate"), bounds=[
    (8, 64), (8, 64), (2, 5), (16, 128), (1e-4, 1e-2)
```

```
])
print("Best Params:", best_params)

# Train models for all fusion types using optimized parameters
build_and_train_model(best_params, "early")
build_and_train_model(best_params, "intermediate")
build_and_train_model(best_params, "late")
```

```
Epoch 1/3
103/103 [==============================] - 5s 16ms/step - loss: 0.8414
- accuracy: 0.5919 - val_loss: 0.8068 - val_accuracy: 0.6087
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7889
- accuracy: 0.6237 - val_loss: 0.7955 - val_accuracy: 0.6142
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7741
- accuracy: 0.6307 - val_loss: 0.7700 - val_accuracy: 0.6376
Epoch 1/3
103/103 [==============================] - 4s 14ms/step - loss: 0.8439
- accuracy: 0.5983 - val_loss: 0.8056 - val_accuracy: 0.6178
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7938
- accuracy: 0.6143 - val_loss: 0.7918 - val_accuracy: 0.6133
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7816
- accuracy: 0.6202 - val_loss: 0.7832 - val_accuracy: 0.6291
Epoch 1/3
103/103 [==============================] - 3s 14ms/step - loss: 0.8330
- accuracy: 0.5958 - val_loss: 0.8012 - val_accuracy: 0.6151
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7866
- accuracy: 0.6230 - val_loss: 0.7766 - val_accuracy: 0.6272
Epoch 3/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7651
- accuracy: 0.6329 - val_loss: 0.7632 - val_accuracy: 0.6364
Epoch 1/3
103/103 [==============================] - 4s 15ms/step - loss: 0.8307
- accuracy: 0.5931 - val_loss: 0.7931 - val_accuracy: 0.6275
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7786
- accuracy: 0.6272 - val_loss: 0.7793 - val_accuracy: 0.6230
Epoch 3/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7718
- accuracy: 0.6294 - val_loss: 0.7679 - val_accuracy: 0.6318
Epoch 1/3
103/103 [==============================] - 3s 14ms/step - loss: 0.8623
- accuracy: 0.5876 - val_loss: 0.8064 - val_accuracy: 0.6178
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7934
- accuracy: 0.6128 - val_loss: 0.7913 - val_accuracy: 0.6175
```

```
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7812
- accuracy: 0.6191 - val_loss: 0.8001 - val_accuracy: 0.6114
Epoch 1/3
103/103 [==============================] - 3s 13ms/step - loss: 0.8475
- accuracy: 0.5916 - val_loss: 0.8053 - val_accuracy: 0.6047
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7827
- accuracy: 0.6248 - val_loss: 0.7759 - val_accuracy: 0.6361
Epoch 3/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7706
- accuracy: 0.6331 - val_loss: 0.7903 - val_accuracy: 0.6212
Epoch 1/3
103/103 [==============================] - 4s 15ms/step - loss: 0.8266
- accuracy: 0.6003 - val_loss: 0.8052 - val_accuracy: 0.6233
Epoch 2/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7797
- accuracy: 0.6270 - val_loss: 0.7946 - val_accuracy: 0.6336
Epoch 3/3
103/103 [==============================] - 1s 11ms/step - loss: 0.7718
- accuracy: 0.6284 - val_loss: 0.7677 - val_accuracy: 0.6440
Epoch 1/3
103/103 [==============================] - 3s 14ms/step - loss: 0.8246
- accuracy: 0.5999 - val_loss: 0.7857 - val_accuracy: 0.6303
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7823
- accuracy: 0.6253 - val_loss: 0.7650 - val_accuracy: 0.6437
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7564
- accuracy: 0.6442 - val_loss: 0.7445 - val_accuracy: 0.6406
Epoch 1/3
103/103 [==============================] - 4s 15ms/step - loss: 0.8468
- accuracy: 0.5939 - val_loss: 0.8002 - val_accuracy: 0.6199
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7913
- accuracy: 0.6202 - val_loss: 0.7966 - val_accuracy: 0.6157
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7810
- accuracy: 0.6275 - val_loss: 0.7807 - val_accuracy: 0.6333
Epoch 1/3
103/103 [==============================] - 3s 13ms/step - loss: 0.8618
- accuracy: 0.5806 - val_loss: 0.8189 - val_accuracy: 0.5935
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7940
- accuracy: 0.6164 - val_loss: 0.7963 - val_accuracy: 0.6175
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7824
- accuracy: 0.6261 - val_loss: 0.7857 - val_accuracy: 0.6242
Epoch 1/3
```

```
103/103 [==============================] - 4s 16ms/step - loss: 0.8285
- accuracy: 0.6036 - val_loss: 0.7946 - val_accuracy: 0.6199
Epoch 2/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7871
- accuracy: 0.6210 - val_loss: 0.7761 - val_accuracy: 0.6291
Epoch 3/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7713
- accuracy: 0.6309 - val_loss: 0.7559 - val_accuracy: 0.6452
Epoch 1/3
103/103 [==============================] - 3s 13ms/step - loss: 0.8381
- accuracy: 0.5926 - val_loss: 0.7883 - val_accuracy: 0.6239
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7804
- accuracy: 0.6274 - val_loss: 0.7890 - val_accuracy: 0.6160
Epoch 3/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7568
- accuracy: 0.6391 - val_loss: 0.7487 - val_accuracy: 0.6421
Epoch 1/3
103/103 [==============================] - 3s 13ms/step - loss: 0.8629
- accuracy: 0.5863 - val_loss: 0.8043 - val_accuracy: 0.6224
Epoch 2/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7902
- accuracy: 0.6190 - val_loss: 0.7842 - val_accuracy: 0.6209
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7805
- accuracy: 0.6231 - val_loss: 0.7828 - val_accuracy: 0.6224
Epoch 1/3
103/103 [==============================] - 4s 15ms/step - loss: 0.8504
- accuracy: 0.5930 - val_loss: 0.8003 - val_accuracy: 0.6154
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7926
- accuracy: 0.6126 - val_loss: 0.7900 - val_accuracy: 0.6090
Epoch 3/3
103/103 [==============================] - 1s 8ms/step - loss: 0.7805
- accuracy: 0.6212 - val_loss: 0.7874 - val_accuracy: 0.6136
Epoch 1/3
103/103 [==============================] - 3s 15ms/step - loss: 0.8605
- accuracy: 0.5824 - val_loss: 0.7992 - val_accuracy: 0.6251
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7863
- accuracy: 0.6193 - val_loss: 0.7837 - val_accuracy: 0.6282
Epoch 3/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7717
- accuracy: 0.6345 - val_loss: 0.7713 - val_accuracy: 0.6306
Epoch 1/3
103/103 [==============================] - 4s 15ms/step - loss: 0.8292
- accuracy: 0.5933 - val_loss: 0.7968 - val_accuracy: 0.6199
Epoch 2/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7854
```

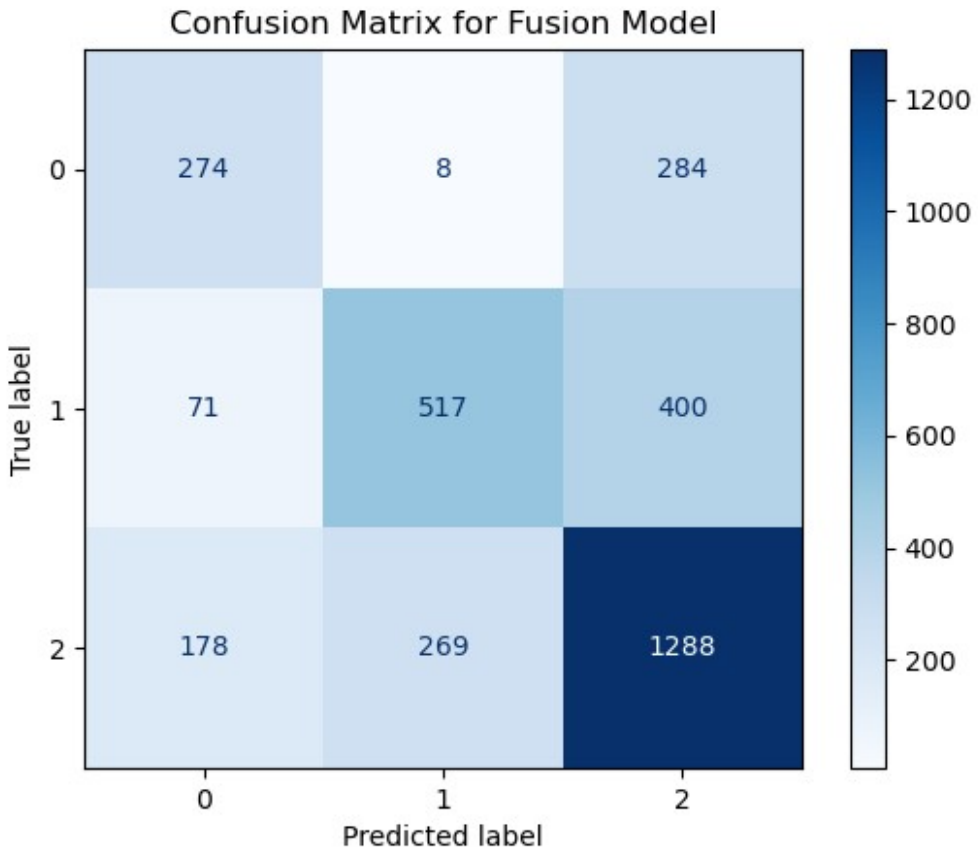
```
- accuracy: 0.6237 - val_loss: 0.7827 - val_accuracy: 0.6272
Epoch 3/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7761
- accuracy: 0.6301 - val_loss: 0.7752 - val_accuracy: 0.6397
Best Params: [2.36554469e+01 5.28159226e+01 3.10703227e+00
5.89867417e+01
 7.35422489e-03]
Epoch 1/3
103/103 [==============================] - 1s 4ms/step - loss: 0.8534
- accuracy: 0.5865 - val_loss: 0.8093 - val_accuracy: 0.6111
Epoch 2/3
103/103 [==============================] - 0s 3ms/step - loss: 0.7967
- accuracy: 0.6238 - val_loss: 0.7909 - val_accuracy: 0.6409
Epoch 3/3
103/103 [==============================] - 0s 2ms/step - loss: 0.7859
- accuracy: 0.6313 - val_loss: 0.7846 - val_accuracy: 0.6321
Epoch 1/3
103/103 [==============================] - 4s 16ms/step - loss: 0.8237
- accuracy: 0.5994 - val_loss: 0.7905 - val_accuracy: 0.6181
Epoch 2/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7805
- accuracy: 0.6205 - val_loss: 0.7744 - val_accuracy: 0.6282
Epoch 3/3
103/103 [==============================] - 1s 11ms/step - loss: 0.7696
- accuracy: 0.6331 - val_loss: 0.7685 - val_accuracy: 0.6257
Epoch 1/3
103/103 [==============================] - 4s 14ms/step - loss: 0.8385
- accuracy: 0.5837 - val_loss: 0.8173 - val_accuracy: 0.6041
Epoch 2/3
103/103 [==============================] - 1s 9ms/step - loss: 0.7993
- accuracy: 0.6096 - val_loss: 0.7948 - val_accuracy: 0.6260
Epoch 3/3
103/103 [==============================] - 1s 10ms/step - loss: 0.7842
- accuracy: 0.6196 - val_loss: 0.7759 - val_accuracy: 0.6120

<keras.engine.functional.Functional at 0x17bbe8408e0>

from tensorflow.keras.models import load_model

model = load_model('lln_cnn_early_fusion_model.h5')

from tensorflow.keras.models import load_model
import numpy as np

# Load models
models = {
    "early": load_model("lln_cnn_early_fusion_model.h5"),
    "intermediate":
load_model("lln_cnn_intermediate_fusion_model.h5"),
    "late": load_model("lln_cnn_late_fusion_model.h5")
```

```
}

# Evaluate each model
for name, model in models.items():
    loss, accuracy = model.evaluate([X_lln_test, X_cnn_test], y_test,
verbose=0)
    print(f"{name.capitalize()} Fusion Accuracy: {accuracy:.4f}")
```

```
Early Fusion Accuracy: 0.6321
Intermediate Fusion Accuracy: 0.6257
Late Fusion Accuracy: 0.6120
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

y_pred = model.predict([X_lln_test, X_cnn_test])
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix for Fusion Model")
plt.show()
```

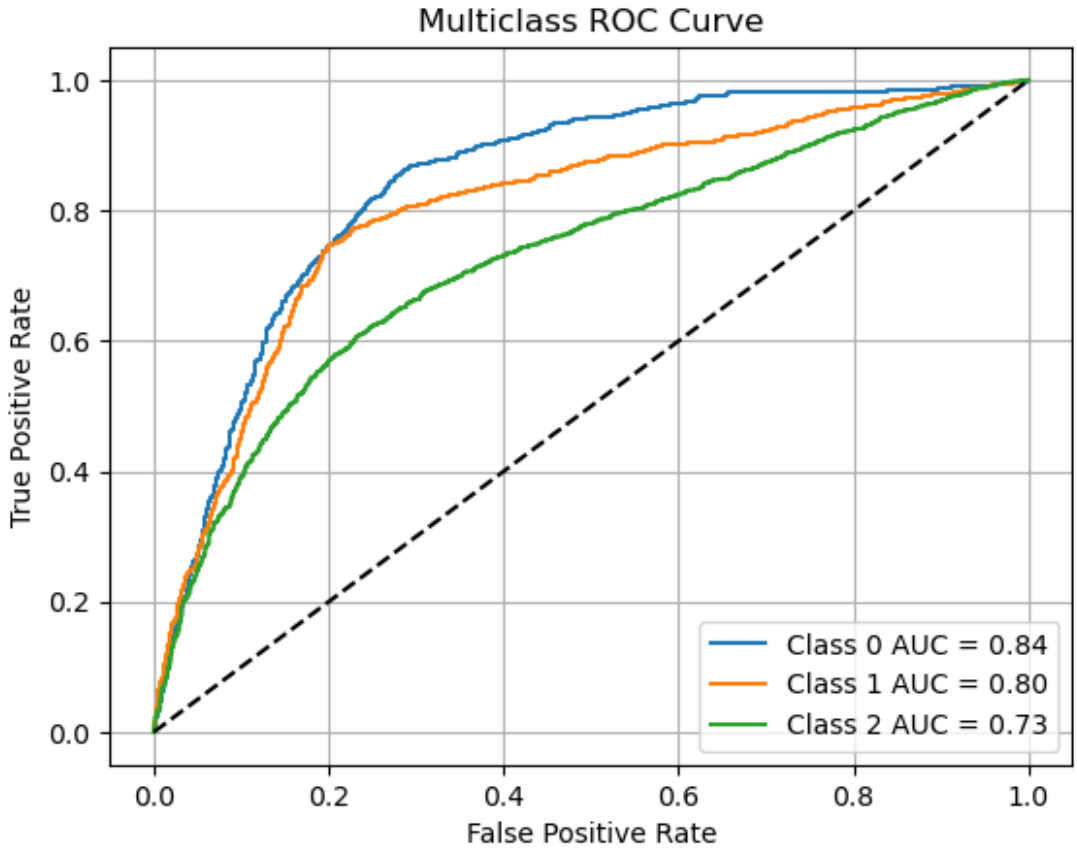
```
103/103 [==============================] - 0s 1ms/step
```




```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import RocCurveDisplay

# Binarize labels for multi-class ROC
y_test_bin = label_binarize(y_test, classes=np.unique(y_test))
y_score = model.predict([X_lln_test, X_cnn_test])

fpr = dict()
tpr = dict()
roc_auc = dict()
n_classes = y_test_bin.shape[1]

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC for each class
plt.figure()
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'Class {i} AUC =
{roc_auc[i]:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multiclass ROC Curve')
plt.legend()
plt.grid()
plt.show()
```

```
103/103 [==============================] - 0s 1ms/step
```




```
from sklearn.metrics import f1_score, average_precision_score

f1 = f1_score(y_test, y_pred_classes, average='weighted')
auprc = average_precision_score(y_test_bin, y_score,
average='weighted')

print(f"Weighted F1-score: {f1:.4f}")
print(f"Weighted AUPRC: {auprc:.4f}")
```

```
Weighted F1-score: 0.6275
Weighted AUPRC: 0.6709
```